

Aim

To implement the Brute Force Pattern Matching algorithm for searching keywords in a document and analyze its efficiency.

Problem Statement

A document editor uses brute-force pattern matching to locate keywords in large research papers.

- a) Explain how the pattern matching process works.
 - b) Analyze the time complexity of this approach.
 - c) Justify the need for more efficient algorithms.
-

Theory

Pattern matching is the process of searching for a smaller string (pattern) inside a larger string (text).

In the Brute Force method, the algorithm compares the pattern with every possible position in the text until a match is found or the entire text has been checked.

It is the simplest string matching algorithm because it does not perform any preprocessing.

(a) Pattern Matching Process

1. Read the input text.
 2. Read the keyword (pattern).
 3. Compare the pattern with the text starting from the first character.
 4. If all characters match, report the position.
 5. Otherwise, shift the pattern by one position and repeat.
 6. Continue until the end of the text.
-

Algorithm

1. Read the text.
 2. Read the pattern.
 3. Calculate the lengths of the text and pattern.
 4. Compare each character of the pattern with the corresponding characters in the text.
 5. If all characters match, print the starting index.
 6. Continue until all positions are checked.
 7. If no match exists, display "Pattern not found."
-

(b) Time Complexity Analysis

Let

- **n** = length of the text
- **m** = length of the pattern

Best Case

The first comparison matches immediately.

$$O(m)$$

Average Case

Several comparisons are made before finding the pattern.

$$O(nm)$$

Worst Case

Every possible position is checked and almost every character is compared.

$$O(nm)$$

Space Complexity

Only a few variables are used.

$O(1)$

(c) Need for More Efficient Algorithms

Although Brute Force Pattern Matching is easy to implement, it becomes inefficient for very large documents because it compares characters repeatedly.

More advanced algorithms improve performance by avoiding unnecessary comparisons.

Examples include:

- Knuth-Morris-Pratt (KMP)
- Boyer-Moore
- Rabin-Karp

These algorithms significantly reduce the number of comparisons and are commonly used in search engines, text editors, plagiarism detection, and document indexing.

Source Code

```
#include <stdio.h>
#include <string.h>

int main()
{
    char text[200], pattern[100];
    int i, j;
    int found = 0;

    printf("Enter the text: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter the pattern to search: ");
    fgets(pattern, sizeof(pattern), stdin);

    // Remove newline characters
    text[strcspn(text, "\n")] = '\0';
    pattern[strcspn(pattern, "\n")] = '\0';
```

```
int n = strlen(text);
int m = strlen(pattern);

// Brute Force Pattern Matching
for(i = 0; i <= n - m; i++)
{
    for(j = 0; j < m; j++)
    {
        if(text[i + j] != pattern[j])
            break;
    }

    if(j == m)
    {
        printf("\nPattern found at position %d\n", i);
        found = 1;
    }
}

if(!found)
    printf("\nPattern not found.\n");

return 0;
}
```

Output

Case 1

Enter the text: Design and Analysis of Algorithms
Enter the pattern to search: Analysis

Pattern found at position 11

Case 2

Enter the text: Computer Science
Enter the pattern to search: Cyber

Pattern not found.

Analysis

The Brute Force Pattern Matching algorithm checks every possible position of the pattern in the text.

It is simple to understand and implement but performs many unnecessary comparisons, making it inefficient for large datasets.

Its performance decreases as the text and pattern sizes increase.

Advantages

- Simple implementation.
 - No preprocessing required.
 - Works for all input strings.
-

Disadvantages

- Slow for large documents.
 - Performs repeated character comparisons.
 - Higher execution time compared to optimized algorithms.
-

Applications

- Document Editors
 - Text Search
 - Search Engines
 - DNA Sequence Matching
 - Plagiarism Detection
 - Log File Analysis
-

Result

The Brute Force Pattern Matching algorithm was successfully implemented in C. It correctly locates the keyword within the given text. The algorithm has a **worst-case time complexity of $O(nm)$** and a **space complexity of $O(1)$** . While easy to implement, more efficient algorithms

such as **KMP**, **Boyer-Moore**, and **Rabin-Karp** are preferred for large documents because they reduce unnecessary comparisons.